

2025学年秋季学期

《人工智能导论A》第一次习题课

任课教师：於俊

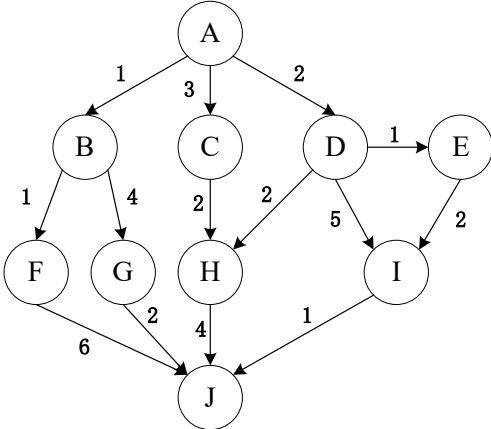
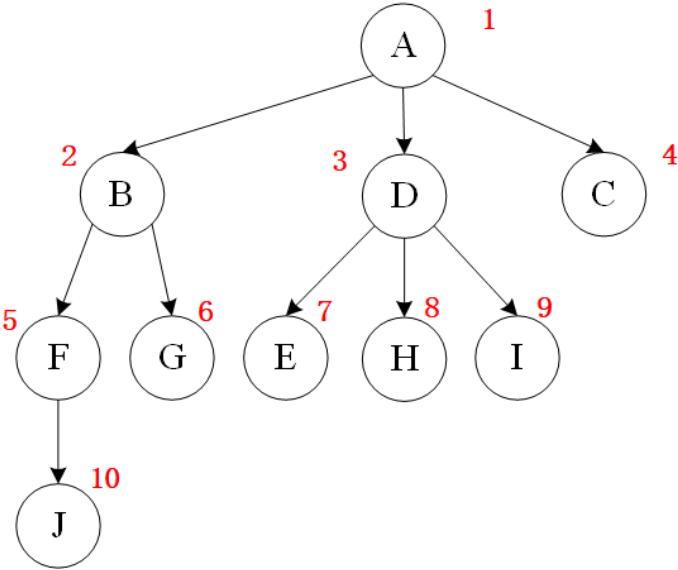
助教：刘凌峰

10/15/2025 10:11:45 AM

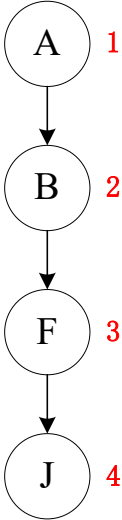
1、如图所示，假设每个结点代表一个状态，结点之间的箭头表示状态转移关系，箭头旁的数字表示状态转移的代价。若使用以下搜索算法寻找从状态 A 到状态 J 的路径，请画出算法终止（找到第一条路径）时的搜索树，并写明结点的扩展顺序，以及找到的路径。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点：

- (1) 基于树搜索的广度优先搜索
- (2) 基于图搜索的深度优先搜索

(1) 找到的路径：A→B→F→J



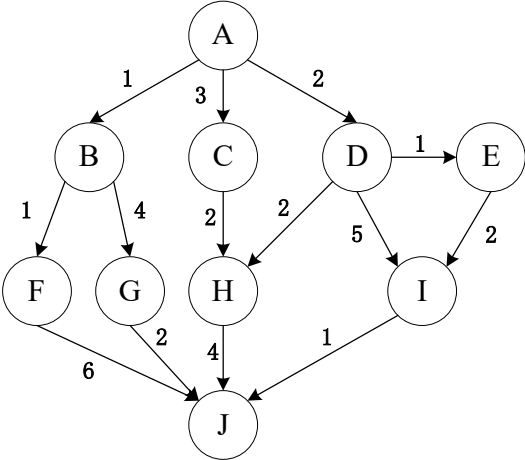
(2) 找到的路径：A→B→F→J



2、针对上一问题，此时给定每个状态的启发函数如表所示。若仍以状态 A 为初始状态、状态 J 为终止状态，请分别使用以下算法求解从 A 到 J 的路径，并画出搜索树。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点：

(1) 基于树搜索的贪婪最佳优先搜索

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0



3.2.1 启发函数与评价函数

启发信息通常被形式化为一个关于结点的函数 $h(n)$ ，其用于估计结点 n 距离达成目标还需付出多少代价，这个函数往往被称为启发函数 (heuristic function)。由于启发函数是用来估计结点之间的距离，因此它通常是非负的。但正如在后文中将看到的，若启发函数存在一些负值

对于边缘集合中的任意结点 n ，函数 $f(n)$ 决定了搜索算法扩展结点 n 的优先度，这个函数被称为评价函数 (evaluation function)。我们规定：边缘集合中某个结点的评价函数值越小，其被挑选的优先级越高。在深度优先搜索中，结点 n 的 $f(n)$ 值可定义为该结点深度的倒数；在广度优先搜索中，结点 n 的 $f(n)$ 值可以定义为该结点的深度。

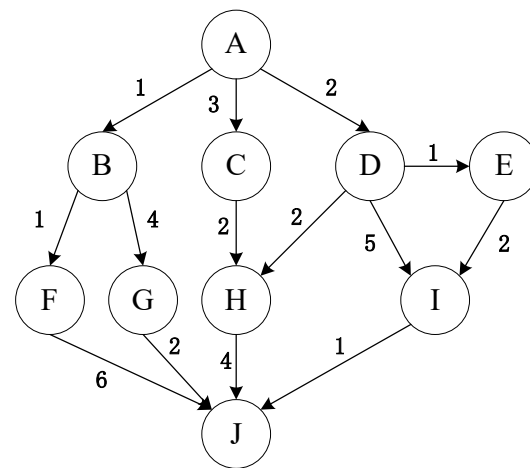
3.2.2 贪婪最佳优先搜索

不难想到，启发信息可以用于计算 $f(n)$ 函数值，以此来影响算法的流程。一种直观的想法是，优先扩展那些距离目标近（启发函数取值小）的结点，即令 $f(n) = h(n)$ ，其中 $h(n)$ 为结点 n 的启发函数值，其用来计算结点 n 到达目标结点所付出代价。令评价函数取值和启发函数取值相等的启发式搜索算法被称为贪婪最佳优先搜索算法 (greedy best-first search)。

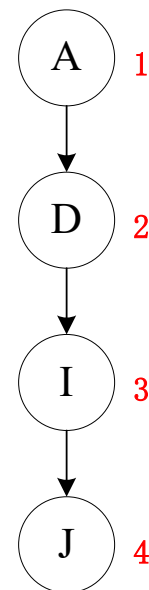
2、针对上一问题，此时给定每个状态的启发函数如表所示。若仍以状态 A 为初始状态、状态 J 为终止状态，请分别使用以下算法求解从 A 到 J 的路径，并画出搜索树。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点：

(1) 基于树搜索的贪婪最佳优先搜索

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0



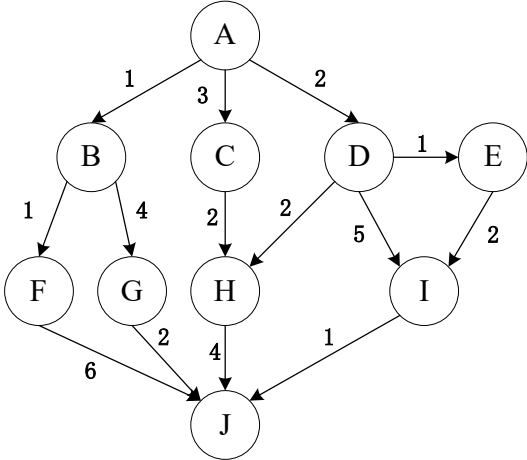
- (1) 贪婪最佳优先搜索 $f(n) = h(n)$ ，优先拓展 $f(n)$ 小的节点
- ① A，此时 $f(B) = 4$ ， $f(C) = 4$ ， $f(D) = 3$ ，拓展 D
- ② $A \rightarrow D$ ，此时 $f(E) = 3$ ， $f(H) = 3$ ， $f(I) = 1$ ，拓展 I
- ③ $A \rightarrow D \rightarrow I$ ，此时 $f(J) = 0$ ，得到路径 $A \rightarrow D \rightarrow I \rightarrow J$ （最佳路径？）



2、针对上一问题，此时给定每个状态的启发函数如表所示。若仍以状态 A 为初始状态、状态 J 为终止状态，请分别使用以下算法求解从 A 到 J 的路径，并画出搜索树。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点：

(2) 基于图搜索的 A* 算法

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0



3.2.3 A* 搜索

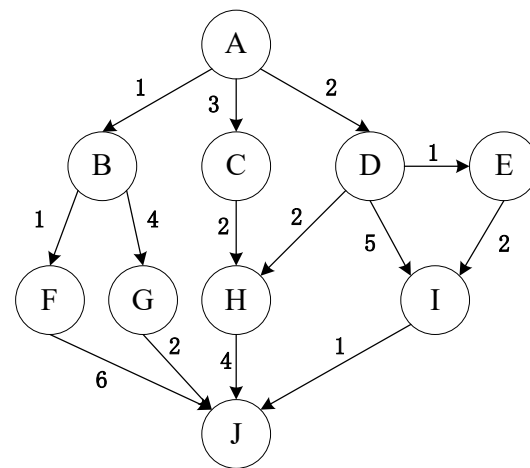
贪婪最佳优先算法的问题在于算法总是贪婪地扩展与目标结点之间代价最小的结点，而不考虑从初始结点到达该结点所需的代价。

为了解决这个问题，一种直观方案可把从起始结点到当前结点的路径代价在评价函数中予以考虑。这样，定义一个结点 n 的函数 $g(n)$ ， $g(n)$ 取值为从起始结点到结点 n 的路径代价，于是评价函数可定义为 $f(n) = g(n) + h(n)$ 。按照这个定义所得到的搜索算法被称为 A* 算法，其评价函数可认为是从起始结点到结点 n 的实际最小代价与从结点 n 到目标结点的后续估计最小代价之和，如下式所示。

2、针对上一问题，此时给定每个状态的启发函数如表所示。若仍以状态 A 为初始状态、状态 J 为终止状态，请分别使用以下算法求解从 A 到 J 的路径，并画出搜索树。若有多个结点拥有相同的扩展优先度，则优先扩展对应路径字典序较小的结点：

(2) 基于图搜索的 A* 算法

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0



(2) A*算法 $f(n) = g(n) + h(n)$ ，优先拓展 $f(n)$ 小的节点

①A，此时 $f(B) = 1 + 4 = 5$ ， $f(C) = 3 + 4 = 7$ ， $f(D) = 2 + 3 = 5$ ，拓展 B

②A→B，此时 $f(F) = 2 + 5 = 7$ ， $f(G) = 5 + 1 = 6$ ，拓展 D

③A→B，A→D，此时 $f(E) = 3 + 3 = 6$ ， $f(H) = 4 + 3 = 7$ ， $f(I) = 8 + 1 = 9$ ，拓展 E

④A→B，A→D→E，此时更新 $f(I) = 5 + 1 = 6$ ，拓展 G

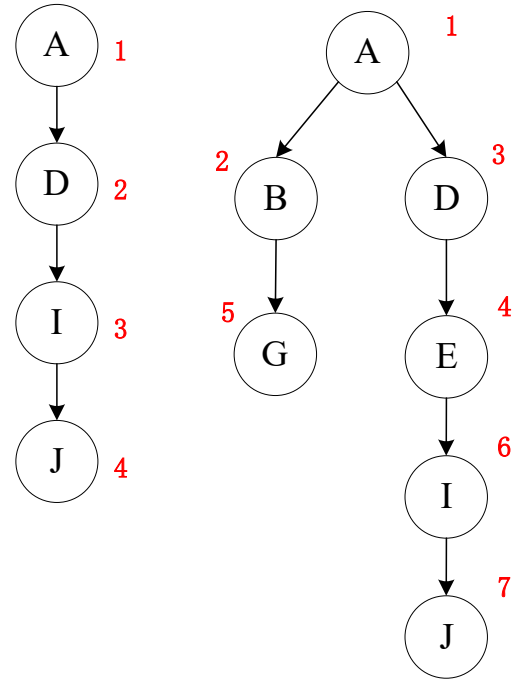
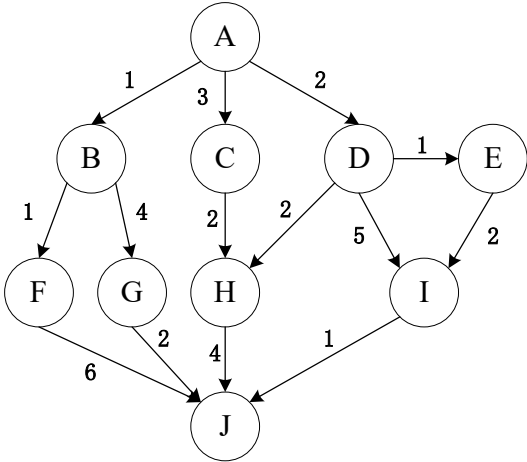
⑤A→B→G，A→D→E，此时 $f(J) = 7 + 0 = 7$ ，拓展 I

⑥A→B→G，A→D→E→I，此时更新 $f(J) = 6 + 0 = 6$ ，得到路径 A→D→E→I→J

2、(2) 基于图搜索的 A* 算法

(3) 请说明：针对 A* 搜索算法，怎样的启发函数才能提高搜索效率？

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0

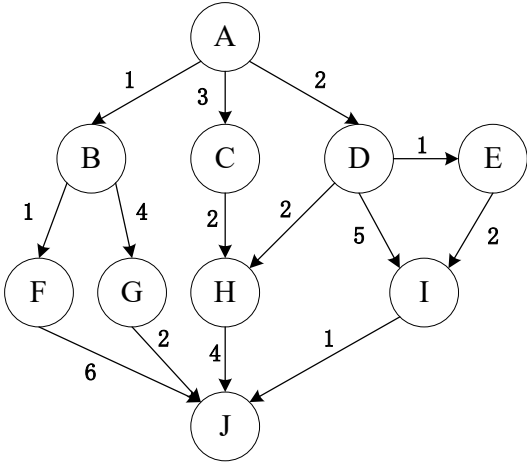


贪婪最佳优先搜索并不具有最优性，A*搜索是在已知信息下同类搜索策略中最优的，额外信息仅包括当前的启发函数时，所有能够保证最优性(即在任意有最短路径的问题中都能找到最短路径)的算法中，A*算法扩展的节点数量是最少的；

启发函数在满足可容性或一致性的基础上,其值越接近当前节点到终止节点的最小代价 $h^*(n)$ ，搜索的效率越高。当启发函数值等于当前节点到终止节点的最小代价时，算法每一步都会朝着最优的方向探索，以 $O(m)$ 的复杂度得到最优解。

- 2、（3）请说明：针对 A* 搜索算法，怎样的启发函数才能提高搜索效率？
- （4）如果想要搜索效率最高，请通过表格列出此时各状态对应的启发函数值。

状态	A	B	C	D	E	F	G	H	I	J
启发函数	5	4	4	3	3	5	1	3	1	0



（3）启发函数在满足可容性或一致性的基础上,其值越接近当前节点到终止节点的最小代价 $h^*(n)$ ，搜索的效率越高。当启发函数值等于当前节点到终止节点的最小代价时，算法每一步都会朝着最优的方向探索，以 $O(m)$ 的复杂度得到最优解。

（4）启发函数值等于当前节点到终止节点的最小代价

状态	A	B	C	D	E	F	G	H	I	J
启发函数	6	6	6	4	3	6	2	4	1	0

玩家MAX

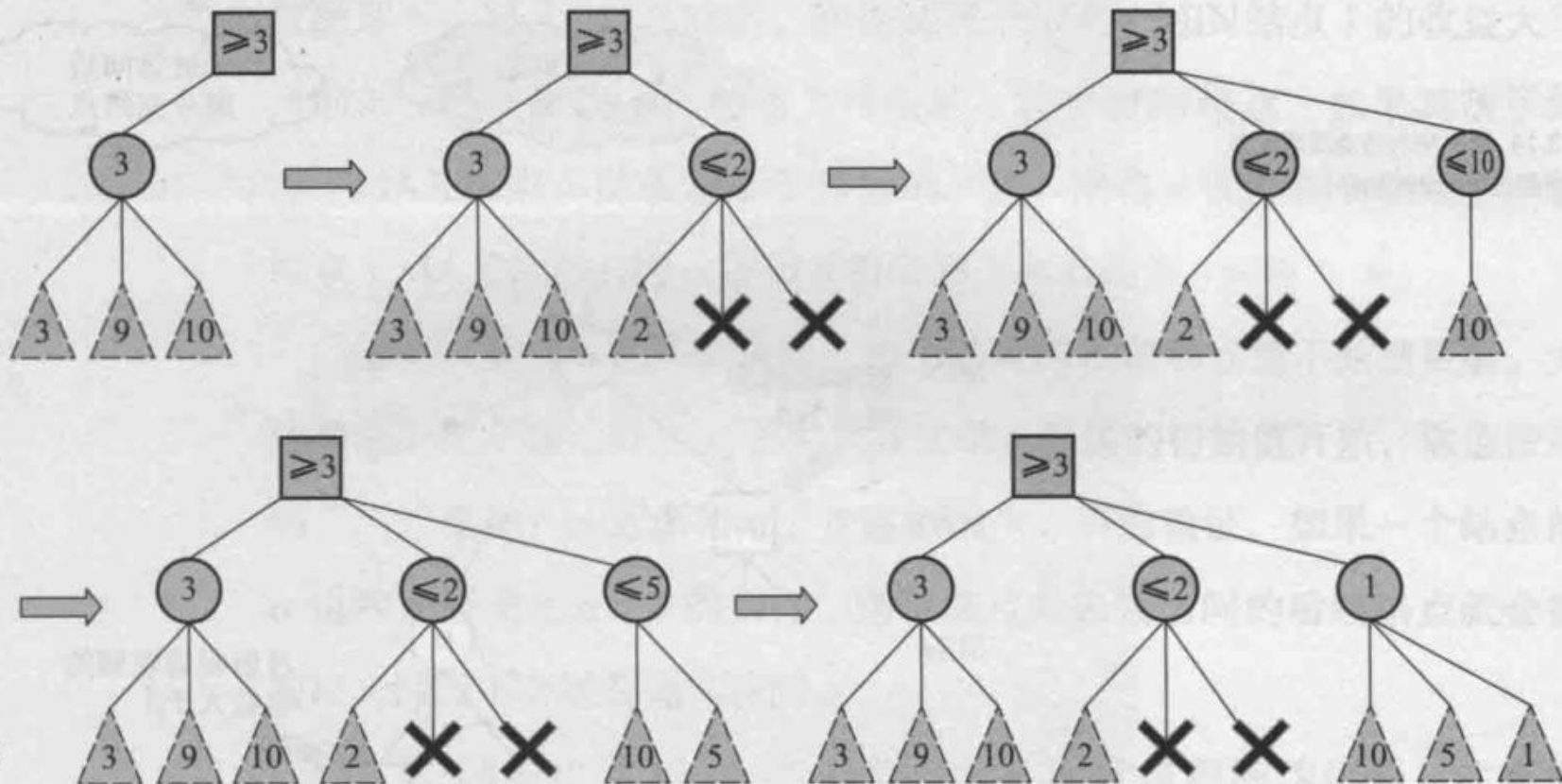
玩家MIN

玩家MAX

玩家MAX

玩家MIN

玩家MAX



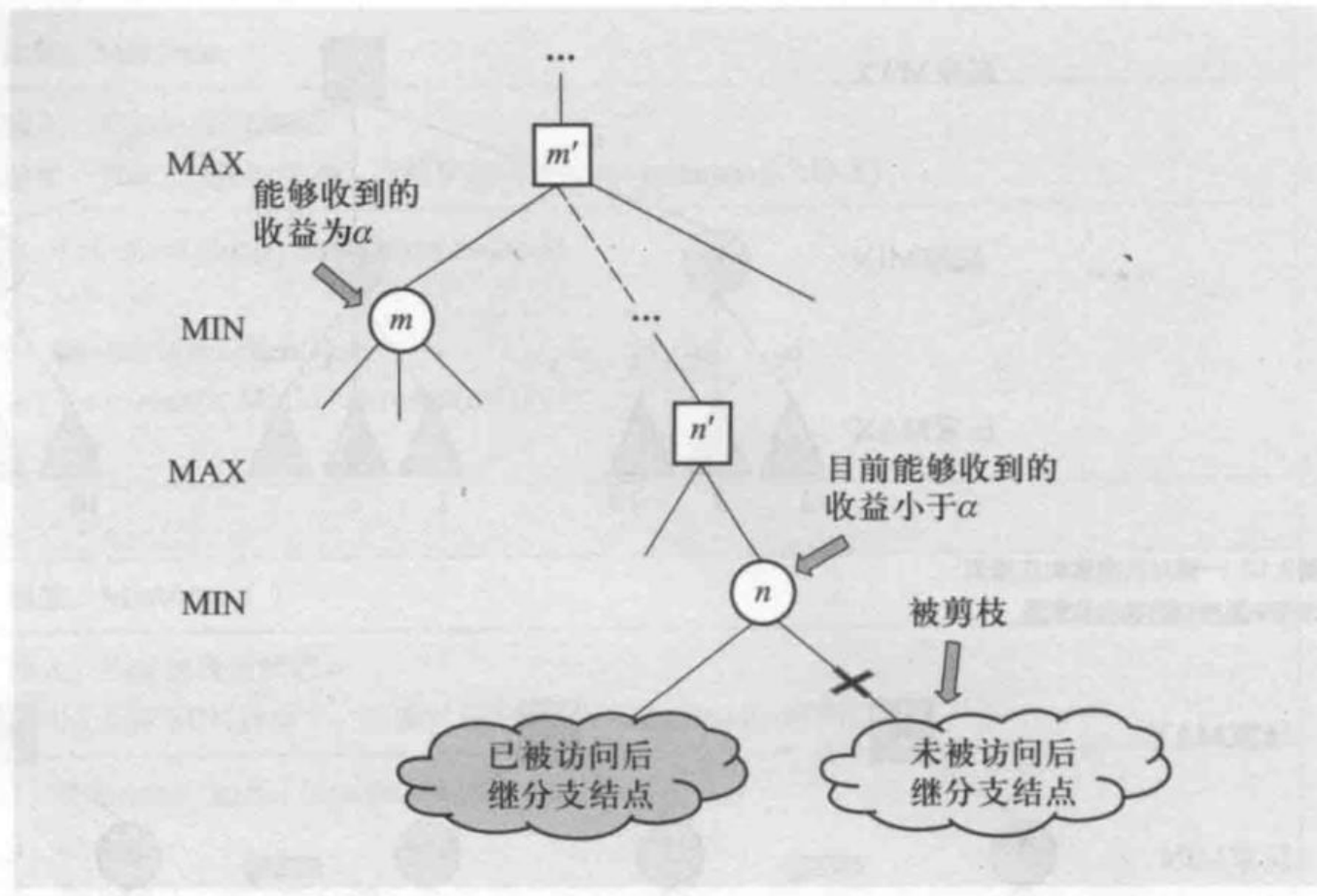


图3.14 基于MIN结点反馈收益进行剪枝 (Alpha剪枝)

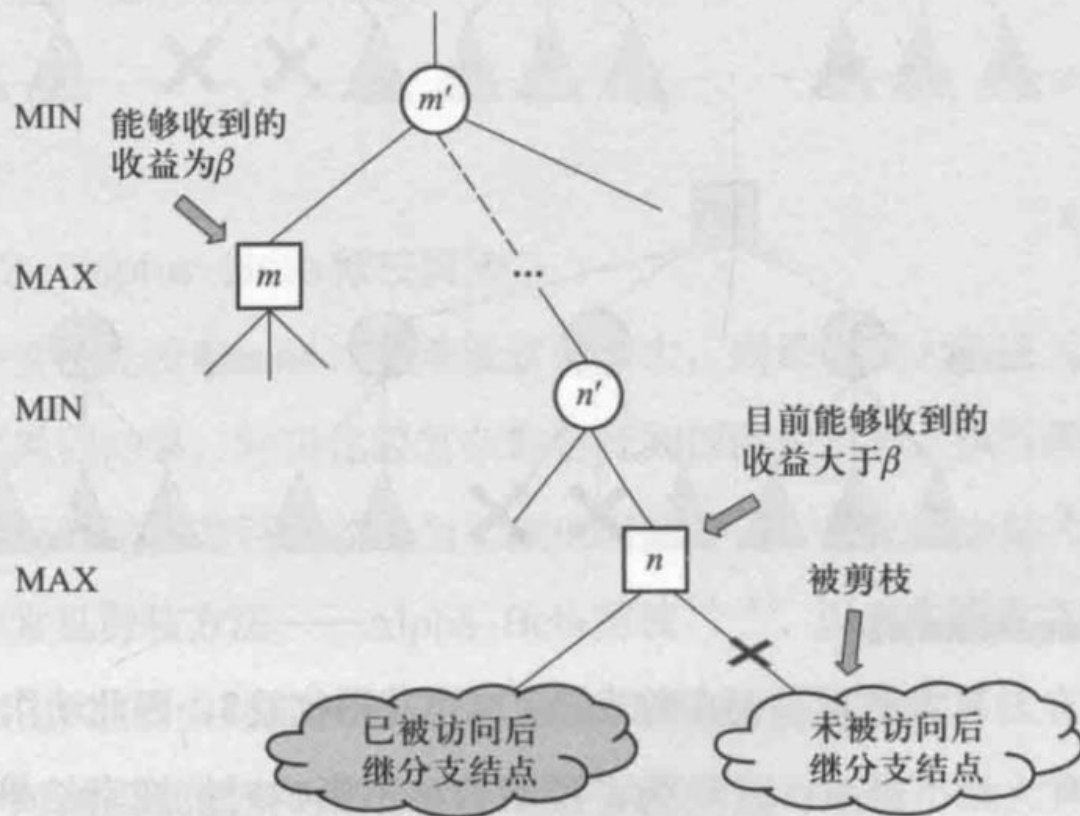


图3.15 基于MAX结点反馈收益
进行剪枝 (Beta剪枝)

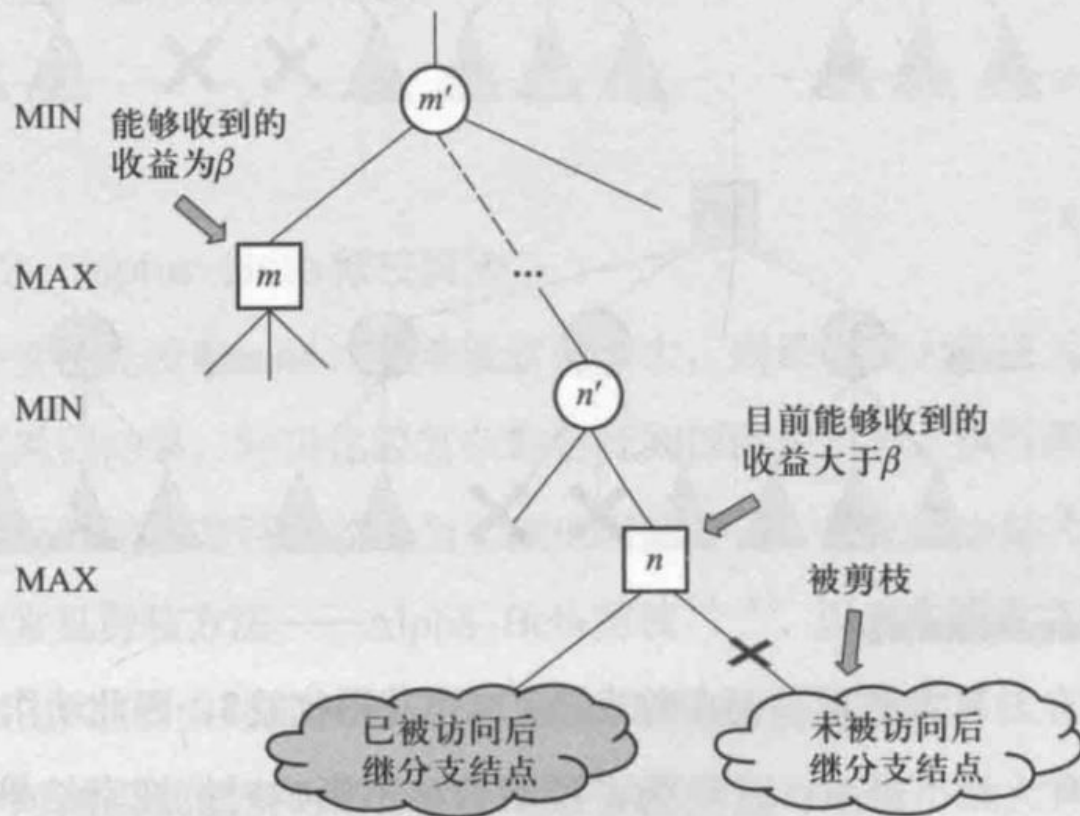


图3.15 基于MAX结点反馈收益
进行剪枝 (Beta剪枝)

Alpha-Beta 剪枝引入了两个参数：alpha 和 beta，它们在搜索过程中动态更新，用于判断是否可以“剪枝”（即跳过某些子树）。

- **Alpha 的含义：**

- Alpha 表示当前 Max 玩家（最大化玩家）在搜索路径上能保证的最低分数（下界）。
- 简单说：它是 Max 已经找到的“最好”选项的下限。如果某个分支的分数低于 alpha，Max 不会选择它，因为 Max 已经有更好的选择了。
- 在搜索过程中，alpha 会随着 Max 节点找到更好分数而向上更新（增加）。

- **Beta 的含义：**

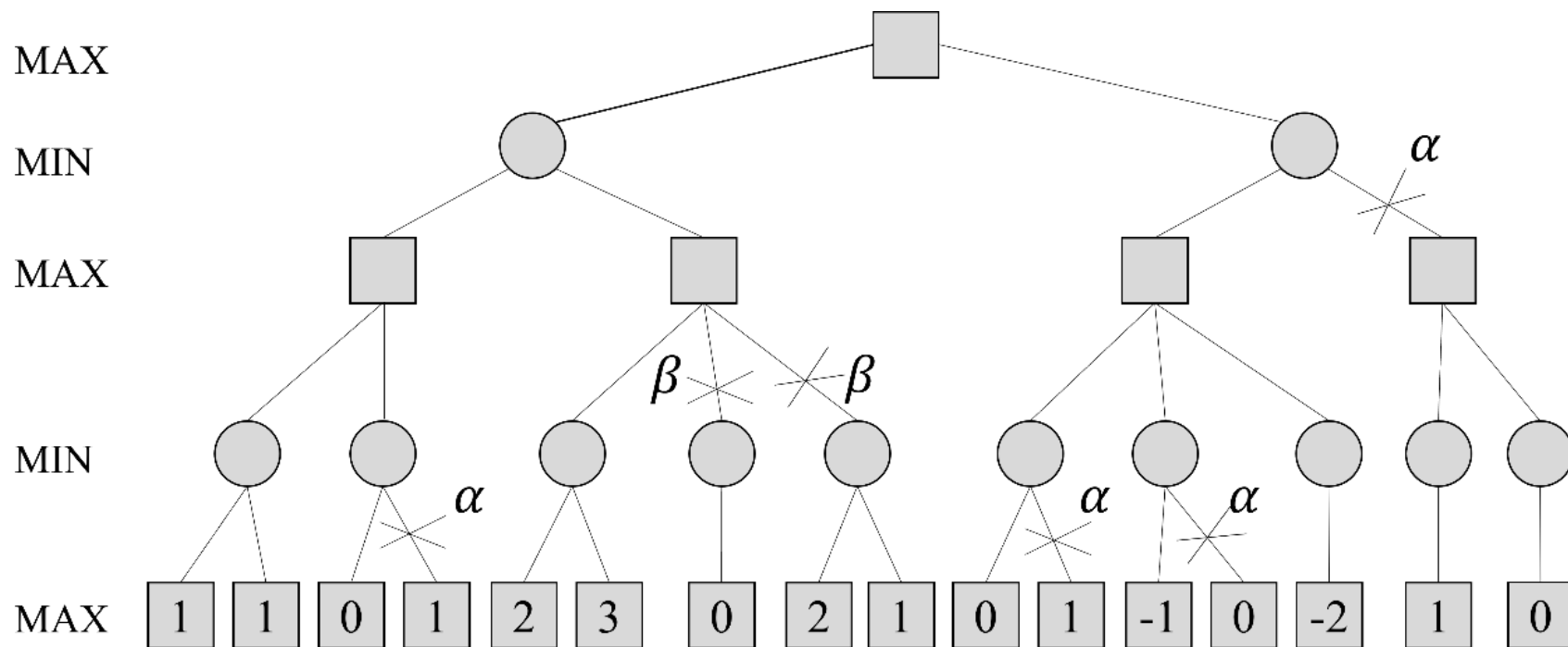
- Beta 表示当前 Min 玩家（最小化玩家）在搜索路径上能保证的最高分数（上界）。
- 简单说：它是 Min 已经找到的“最好”选项的上限。如果某个分支的分数高于 beta，Min 不会选择它，因为 Min 已经有更好的选择了（对 Min 来说，更小的分数更好）。
- 在搜索过程中，beta 会随着 Min 节点找到更好分数而向下更新（减小）。

- **[Alpha, Beta] 区间：**

- 每个节点在进入时都会继承父节点的 [alpha, beta] 值。
- 这个区间表示“可接受的分数范围”：我们只关心分数在 [alpha, beta] 内的分支。如果一个分支的分数超出这个区间，就没有必要继续搜索，因为它不会被选择。
- **剪枝条件：**如果在搜索某个节点时，发现 $\beta \leq \alpha$ （即上界 $\leq$ 下界），则立即剪枝（停止搜索该节点下的子树），因为这个区间已经无效——Max 和 Min 的利益冲突导致这个分支不可能被选中。

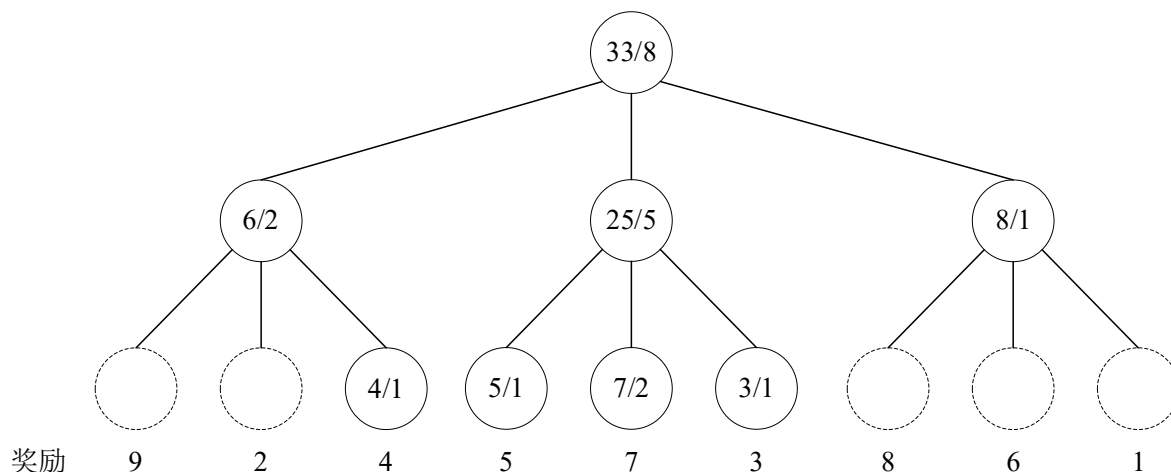
3、如图为一颗最小最大搜索树，可采用 Alpha-Beta 剪枝算法进行对抗搜索。假设对于每个结点的后继结点，算法按照从左向右的方向扩展。同时假设当 Alpha 值等于 Beta 值时，算法不进行剪枝：

对图中所示搜索树进行搜索，请画出在算法结束时搜索树的状态，用“x”符号标出被剪枝的子树，并且对每次剪枝都标记出其为 Alpha 剪枝还是 Beta 剪枝。



4、下图为一个蒙特卡洛树搜索的例子，其中每个叶子结点（终止结点）下标出了该结点对应的奖励。假设执行了若干步骤后，算法的状态如图中所示，结点内的数字分别表示“总奖励/访问次数”，虚线结点表示尚未扩展的结点。

(1) 假设UCB1算法中的超参数 $C=5$ ，请计算并写出算法选择过程经过的路径



UCB1算法的策略是：为每个动作的奖励期望计算一个估计范围优先采用估计范围上限较高的动作。

UCB1算法的策略可以描述为，在第 t 次时选择动作 a_{l_t} 使得（式3.5）取值最大，其中 l_t 由如下式子计算得到：

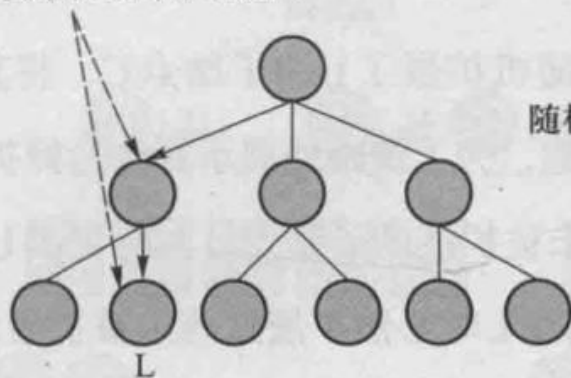
$$l_t = \operatorname{argmax}_i \bar{x}_{i, T_{(i, t-1)}} + C \sqrt{\frac{2 \ln t}{T_{(i, t-1)}}} \quad (\text{式3.5})$$

算法事先不知道每个结点将会得到怎样的代价（或终局分数）分布，只能通过采样式探索来得到计算奖励的样本。由于这个算法利用蒙特卡洛法通过采样来估计每个动作优劣，因此它被称为**蒙特卡洛树搜索**（Monte-Carlo tree search）算法^[6]。

蒙特卡洛树搜索算法分为以下四个步骤。

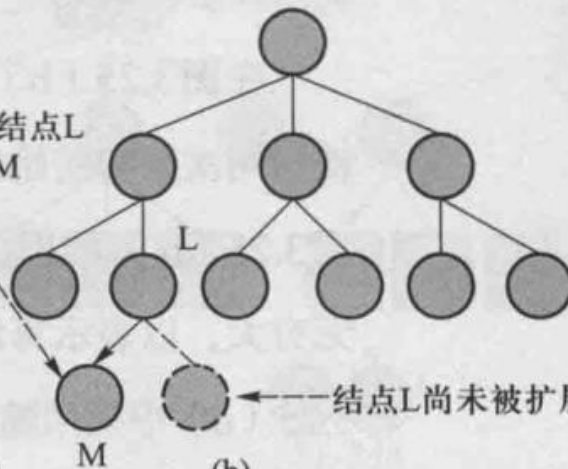
- **选择（selection）**：选择指算法从搜索树的根结点开始，向下递归选择子结点，直至到达叶子结点或者到达具有还未被扩展过的子结点的结点L，如图3.24（a）所示。这个向下递归选择过程可由UCB1算法来实现，在递归选择过程中记录下每个结点被选择次数和每个结点得到的奖励均值。
- **扩展（expansion）**：如果结点L不是一个终止结点（或对抗搜索的终局结点），则随机扩展它的一个未被扩展过的后继边缘结点M，如图3.24（b）所示。
- **模拟（simulation）**：从结点M出发，模拟扩展搜索树，直到找到一个终止结点，如图3.24（c）所示。模拟过程使用的策略和采用UCB1算法实现的选择过程并不相同，前者通常会使用比较简单的策略，例如使用随机策略。
- **反向传播（Back Propagation）**：用模拟所得结果（终止结点的代价或游戏终局分数）回溯更新模拟路径中M以上（含M）结点的奖励均值和被访问次数，如图3.24（d）所示。

UCB1算法递归向下选择结点，
直至当前搜索树的叶子结点L



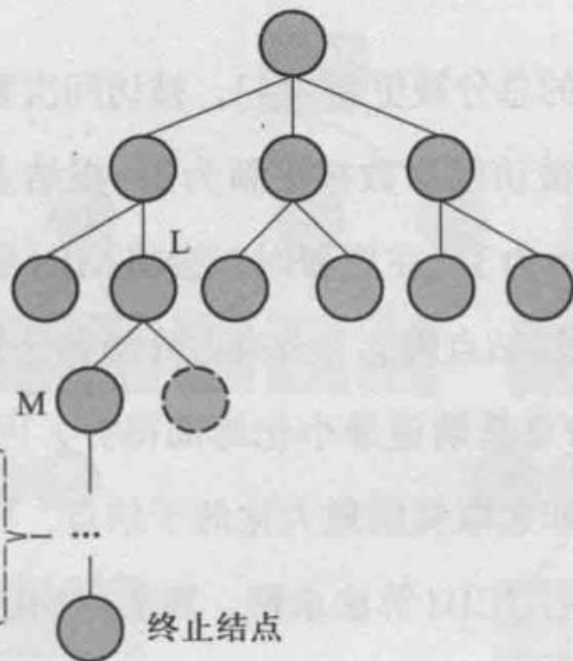
(a)

随机扩展，得到结点L
的孩子结点M



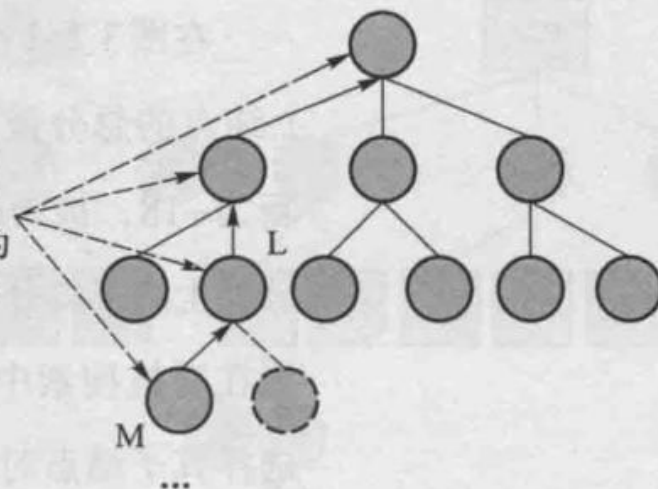
(b)

结点L尚未被扩展的孩子结点



(c)

回溯更新
路径中结点M以上的
结点信息



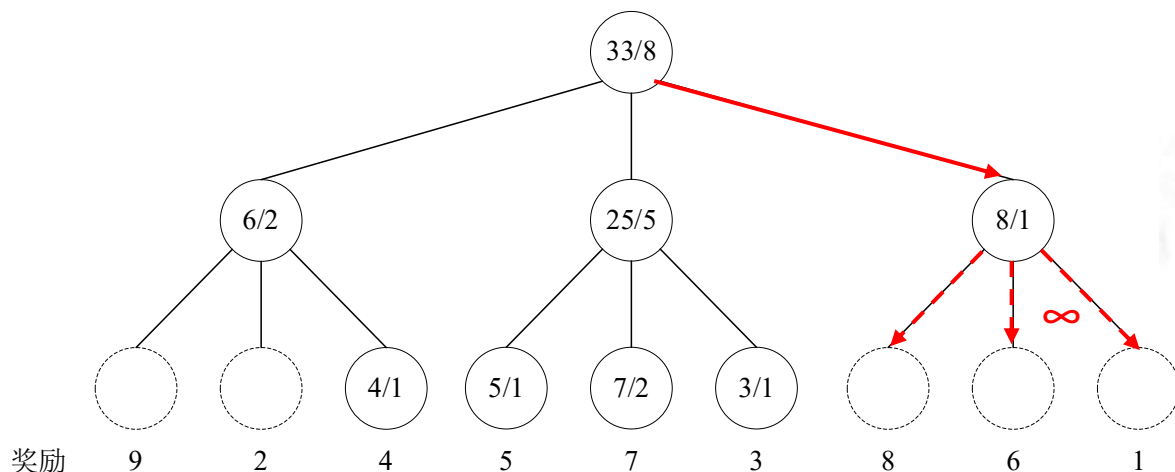
(d)

4、下图为一个蒙特卡洛树搜索的例子，其中每个叶子结点（终止结点）下标出了该结点对应的奖励。假设执行了若干步骤后，算法的状态如图中所示，结点内的数字分别表示“总奖励/访问次数”，虚线结点表示尚未扩展的结点。

(1) 假设UCB1算法中的超参数 $C=5$ ，请计算并写出算法选择过程经过的路径

(1) 第一步三个结点的UCB值： $\frac{6}{2} + 5 \times \sqrt{\frac{2\ln 8}{2}} = 10.21$ ， $\frac{25}{5} + 5 \times \sqrt{\frac{2\ln 8}{5}} = 9.56$ ， $\frac{8}{1} + 5 \times$

$\sqrt{\frac{2\ln 8}{1}} = 18.20$ ，扩展第三个结点；由于该结点并非终止结点，会随机扩展一个未被拓展过的后继结点，相当于UCB值均为无穷大。

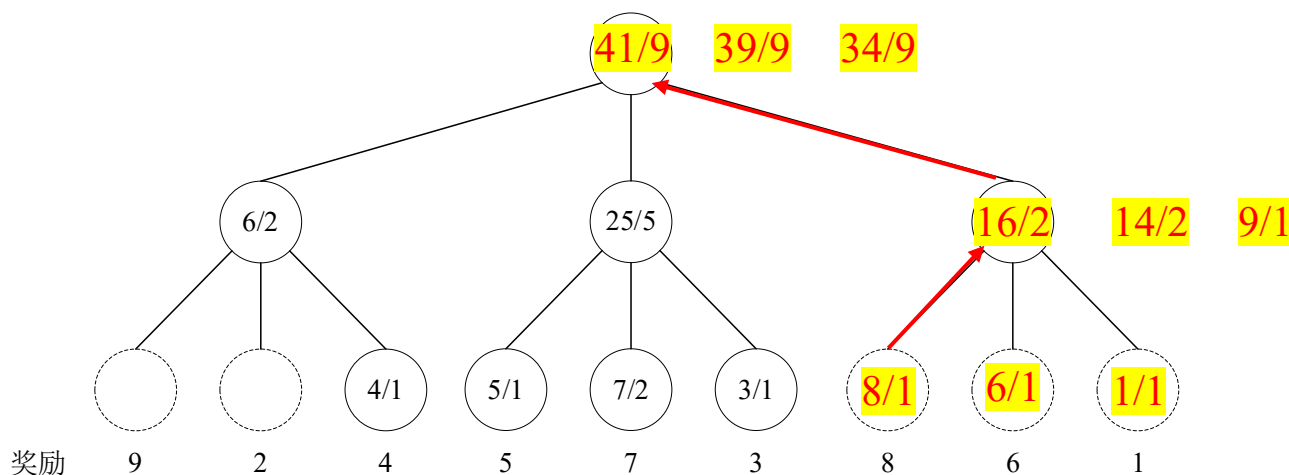


$$l_t = \operatorname{argmax}_i \bar{x}_{i, T_{(i, t-1)}} + C \sqrt{\frac{2 \ln t}{T_{(i, t-1)}}}$$

4、下图为一个蒙特卡洛树搜索的例子，其中每个叶子结点（终止结点）下标出了该结点对应的奖励。假设执行了若干步骤后，算法的状态如图中所示，结点内的数字分别表示“总奖励/访问次数”，虚线结点表示尚未扩展的结点。

(2) 请执行反向传播步骤，并画出完成后的搜索树状态

(3) 为了更高效率的找到奖励最高的叶子节点，有哪些常见的调整策略？



(3) 调整超参数C来加大探索的力度，选择更大的超参数C有利于更快找到奖励最高的叶子节点